

基于文档的多阶段训练LLM用于漏洞类型预测与推理

摘要

本方案旨在通过多阶段训练（监督微调 SFT 与强化学习 RL）微调大型语言模型（LLM），使其能够根据项目的产品文档较为准确地预测其潜在的漏洞类型，并生成合理的推理过程。训练完成后，将对模型在漏洞预测误报率、判别依据合理性等多方面性能进行评估。目标性能需显著超越训练前的基础模型，并与当前优秀的 LLM 进行对比分析；同时，将与 VDocScan 采用的数据处理方法进行对比。计划进行多角度的消融性研究，以验证各设计组件的有效性。后续工作可尝试提升 LLM 指导漏洞验证和利用的能力。初步可行性测试表明任务可行且数据充足。

方法步骤

Step 0: 基础模型选择

建议选择 **Qwen2.5-7B-Instruct** 作为基础模型。该模型作为通用指令模型，在语言理解、逻辑推理、指令遵循上表现全面，能很好地处理各种格式的非代码文档（如产品说明、API 文档等），为后续的针对性训练奠定良好基础。

Step 1: 构建用于监督微调（SFT）的训练数据集

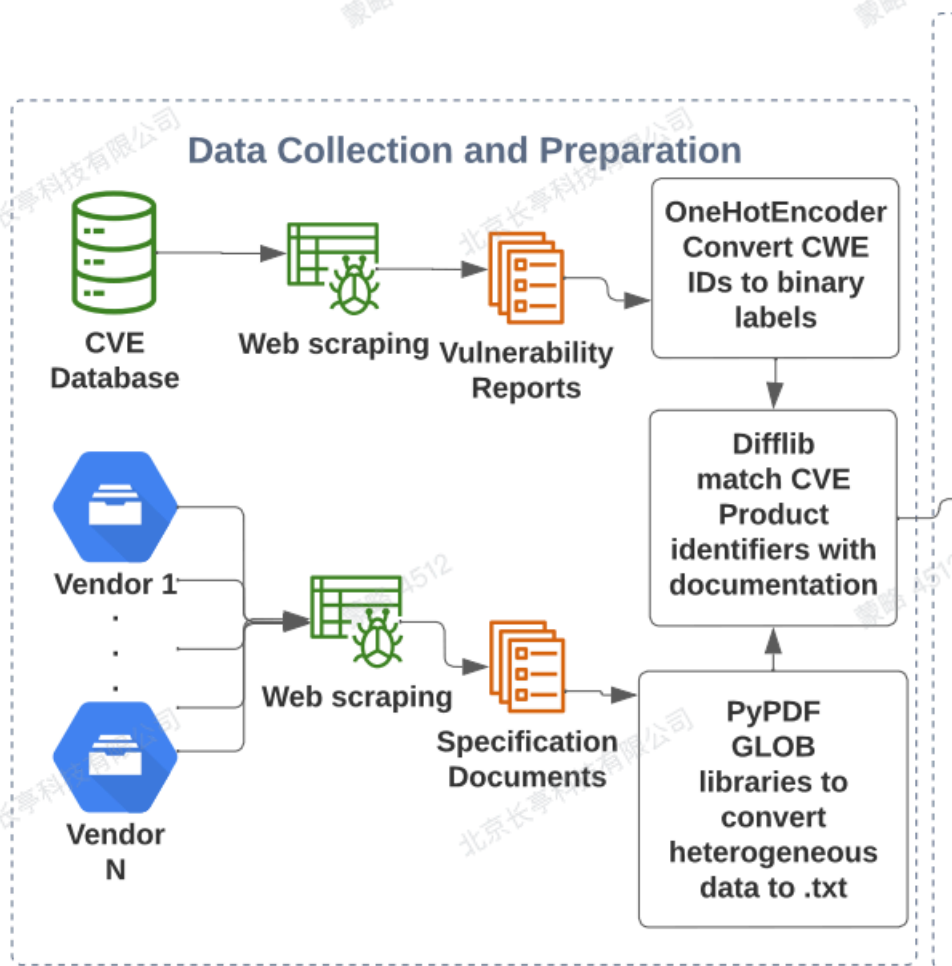
Step 1.1: 数据收集和清洗

1. Github 开源数据（来源：胡宇睿）

- 现有字段：项目名、代码语言、星数、Github 地址、漏洞数量、历史漏洞描述、是否是 Web 系统、是否有 Demo 地址、是否有官方文档、是否有 Dockerfile、描述、demo 地址、官方文档 url、Dockerfile 相对路径。
- 希望增加字段：
 - i. CWE 编号（可直接从历史漏洞描述字段提取）。
 - ii. 文档具体内容（用于预测漏洞的文档，如产品接口、说明文档）。

2. 闭源产品数据——参考 VDocScan 数据集的构建过程（如下图所示）

（网址：<https://zenodo.org/records/7578926>）



- 数据来源: <https://www.cvedetails.com/>
- API接口: <https://www.cvedetails.com/api/v1/swagger-ui/>
- VDocScan数据集介绍:

文件	行数	主要字段说明
VulnerabilityReportsDataset.csv	296,932	产品、供应商、发布日期、CVE ID、CWE信息、产品ID、供应商ID、漏洞类型
ProductsDataset.csv	112,699	产品名称、URL、漏洞数量、产品ID、供应商ID、供应商名称、产品类型
TransformedDataFrameForClassification.csv	2,460	供应商、产品、文档信息, 以及8个漏洞支柱字段 (435、682、703、691、284、693、664、707)

表：VDocScan数据集结构概览

- **漏洞支柱字段**: 435、682、703、691、284、693、664、707 分别对应特定的CWE漏洞类型（如CWE-435为多个行为正确的实体之间的不当交互），值为0/1表示是否存在该类型的漏洞。
- **数据来源**: 数据通过CVE Details的API和爬取产品文档网站获取。
<https://www.cvedetails.com/api/v1/>
- **数据收集方法**:
 - a. **产品对应的漏洞信息**: 调用CVE Details官方API获取。
 - b. **产品的相关文档**: 需要定制化爬虫从不同的供应商网站爬取，可参考VDocScan的爬虫逻辑。具体流程：从CVE数据集中识别产品数量最多的供应商，并识别其网站中具有中心化产品文档/手册链接的页面，然后创建网络爬虫程序，应用所需的安全握手协议，从API/Web服务端点获取数据（HTML或JSON格式）或下载.txt/.pdf格式的文件。

示例: Microsoft Xamarin的产品文档链接: <https://learn.microsoft.com/en-us/previous-versions/xamarin/get-started/what-is-xamarin>

Step 1.2: 数据处理

将收集到的数据，通过映射、清洗和提取，构建为包含以下字段的数据集（参考并扩展VDocScan 的字段设计）：

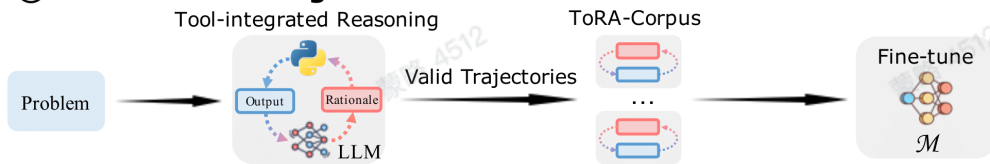
- Vendor（供应商）
- Product（产品）
- Documentation（文档内容）
- Version number（新增，产品版本号）
- Summary（新增，产品漏洞简要描述）
- 435（CWE 435 - 多个行为正确的实体之间的不当交互）
- 682（CWE 682 - 计算错误）
- 703（CWE 703 - 异常条件的检查或处理不当）
- 691（CWE 691 - 控制流管理不足）
- 284（CWE 284 - 访问控制不当）
- 693（CWE 693 - 保护机制失效）
- 664（CWE 664 - 通过其生命周期对资源的控制不当）
- 707（CWE 707 - 未恰当中和）

Step 1.3: 利用强大推理 LLM 构建 SFT 训练数据集（Corpus）

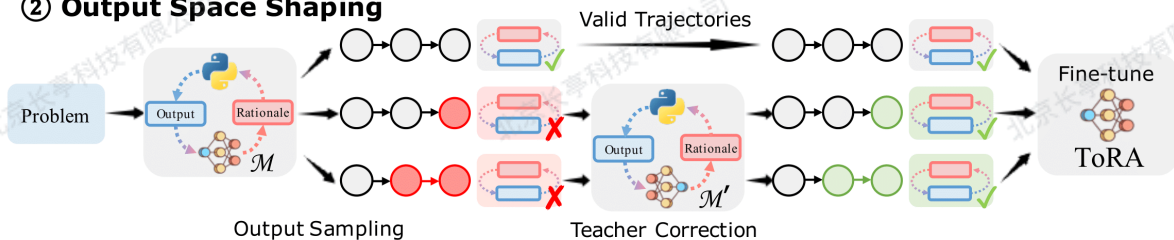
前提条件:

1. Step 1.2 提取出的结构化数据。
2. 若干精心设计并经过测试有效的 Prompts。

① Imitation Learning



② Output Space Shaping



上图：ToRA 流程图

数据格式: Alpaca 格式 `[{"instruction": "...", "input": "...", "output": "..."}]`

- **instruction**: 不是固定的。需精心设计一个 **instruction 模板库**，从不同角度提问（如针对同一样本进行 3-5 个角度的提问）。设计目的：
 - a. 引导模型从多角度理解文档内容（例如：“提取文档中与下述漏洞类型最相关的片段，不超过 2000 词”）。
 - b. 丰富 SFT 数据集，增强模型对文档的理解能力。
- **input**: 包含 Vendor、Product、Documentation、Version number 字段的信息。
- **output**: 必须包含严谨且正确的推理链，由强大推理 LLM（如 GPT-4、Claude-3、DeepSeek-V3.2-Thinking）生成。

推理链生成 Prompt 示例：

“你是一个安全专家。已知产品文档信息如下：...（产品文档）。你需要分析该文档与漏洞...（对应的 Summary、CVE 编号）的关联，并给出一个清晰专业的分析和推理过程。”

数据质量保证：

为提高数据集质量，需要淘汰那些生成质量不佳（如推理混乱、事实错误）的 output 数据。此过程需要安全专家参与标注和筛选。

Step 1.4: 构建评估集

预留 Corpus 的 20% 作为评估集，或基于 VDocScan 的 `TransformedDataFrameForClassification.csv` 数据集自行构建评估集合。必须保证模型在训练过程中未使用过评估数据集。

Step 1.5: 使用 Corpus 构建训练集

将剩余的 80% Corpus 数据用于 SFT 训练。

Step 2: 监督微调（SFT）训练

- 训练框架: Llama-Factory (<https://github.com/hiyouga/LLaMA-Factory>)
- 训练方法: 全参数微调
- 数据划分: 使用 Step 1.5 构建的训练集（80%）进行训练，预留 20% SFT 数据作为 SFT 阶段的验证集，用于监控训练过程，防止过拟合。

Step 3: 强化学习训练（可选）

- 训练框架: Verl (<https://github.com/volcengine/verl>)
- 训练算法: GRPO
- **Step 3.1 奖励函数设计:** 需要同时兼顾**结果准确性**（漏洞预测是否与真实标签一致）和**输出格式要求**（推理过程是否清晰、结构是否合理）。
- **Step 3.2 强化学习训练流程:** 多次迭代训练，每轮训练后评估效果。具体流程可参考 DeepSeekMath-V1 的训练方法 [1]。
- **注:** 在完成高质量的 SFT 训练后，需评估模型性能。若性能已达到满意水平，可考虑简化或省略 RL 步骤，以降低训练复杂度和成本。

Step 4: 评估与消融性研究

4.1 评估基准

使用 Step 1.4 构建的评估集进行测试。评估指标包括：

- 逻辑漏洞预测召回率
- 误报率
- 判别依据（推理过程）的合理性（可通过人工评估或使用另一个 LLM 进行打分）

4.2 对比分析

1. **与基础模型对比:** 对比微调后的模型与原始 Qwen2.5-7B-Instruct 在相同评估集上的性能。
2. **与其他优秀 LLM 对比:** 选择当前先进的、具有较强推理能力的开源或闭源 LLM（如 DeepSeek-V3.2-Thinking、GPT-4o-mini、Claude-3.5-Sonnet 等），在相同评估集上进行 zero-shot/few-shot 对比。
3. **与 VDocscan 方法对比:** 将本方案训练的模型与 VDocScan 论文中（VDocScan 2023）基于传统机器学习（如 TF-IDF+SVM）的方法进行方法上的对比，尤其是数据处理的部分。

4.3 消融性研究

设计三组消融实验，以验证关键设计点的有效性。

1. 消融研究一：思维链（Chain-of-Thought）监督的有效性

- **对照组:** 使用完整的 (instruction, input, output) 三元组进行训练，其中 output 是详细的推理链 + 漏洞预测。
- **实验组:** 将 output 简化为仅包含最终的漏洞预测标签（即去掉推理过程）。
- **验证目标:** 验证思维链监督对于模型学会“推理”而非单纯“匹配”的关键作用。

2. 消融研究二：指令多样性的影响

- **对照组:** 使用设计的“多角度指令模板库”为每个 input 生成多个训练样本。

- **实验组：**使用单一的、固定的指令模板（例如：“请根据以下文档分析可能存在的漏洞。”）为每个 input 生成训练样本。
- **验证目标：**验证“指令增强”是否能提升模型对多样化产品文档的理解能力和泛化能力。

3. 消融研究三：输出格式的影响

- **对照组：**输出为结构化的推理链（例如：先分析功能点 A，关联到潜在风险 B，因此可能属于 CWE-X）。
- **实验组：**输出为自由格式、无固定结构的纯文本分析。
- **验证目标：**验证结构化输出是否有利于模型生成更清晰、更易于评估的推理，并可能提升特定指标（如 CWE 编号）的抽取精度。

Step 5: 完成项目整理与论文写作

整理所有实验数据、代码和模型，按照学术论文规范撰写论文，包括引言、相关工作、方法、实验、结果分析、消融研究、结论与未来工作等部分。

本地可行性测试及测试结果

Q1: 有没有可能让模型学会从文档中进行漏洞预测？现有LLM对漏洞预测的准确性如何？

结论：完全有可能，但对SFT训练数据集和instruction format的质量有很高的要求。现有LLM预测漏洞的准确性较低，且对文档数据和prompt有很高的要求。

测试结果：使用DeepSeek-V3.2 thinking进行3个阶段的测试：

1. **第一阶段：**漏洞相关程度较低产品文档 + zero-shot prompt。结果：10个产品文档中预测成功0个。
2. **第二阶段：**漏洞相关程度较低产品文档 + few-shot prompt。结果：10个产品文档中预测成功1个。
3. **第三阶段：**漏洞相关程度较高的产品文档 + few-shot prompt。结果：10个产品文档中预测成功2个。

Q2: 有没有足够多的我们需要的数据？

结论：有，但需要时间去收集，尤其是文档信息的收集，工作量不小。

测试方法：对CVE Details API进行了调用测试，确认可以获取大量漏洞和产品信息。

总结

本方案旨在通过多阶段训练（SFT与RL）微调Qwen2.5-7B-Instruct，使其能根据项目文档推理预测潜在逻辑漏洞类型，性能目标需超越VDocScan及基础模型。核心在于构建高质量的SFT数据集，其指令需引导多维度分析，输出为严谨推理链。利用开源Github数据与扩建的VDocScan闭源数据，经清洗、映射、格式化后训练。最终通过严格评估（召回率、误报率、推理合理性等）验证模型有效性，并计划进行消融研究与对比分析。初步可行性测试表明任务可行且数据充足。

参考文献&链接

1. VDocScan: Document-driven Vulnerability Discovery. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*.
<https://doi.org/10.1109/ICSE48619.2023.00143>
2. VDocScan Dataset. <https://zenodo.org/records/7578909>
3. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. *arXiv preprint arXiv:2402.03300*. <https://arxiv.org/abs/2402.03300>
4. ToRA: A Tool-Integrated Reasoning Agent for Mathematical Problem Solving. *arXiv preprint arXiv:2309.17452*. <https://arxiv.org/abs/2309.17452>
5. LLaMA-Factory. <https://github.com/hiyouga/LLaMA-Factory>
6. Verl: Volcengine Reinforcement Learning Framework.
<https://github.com/volcengine/verl>